

DATABASE SYSTEMS PROJECT

Streaming Service DB

A MySQL-powered GUI application for browsing movies & series by genre, type, and rating

Problem Statement

Content Overload:

Users struggle to browse large streaming libraries without smart filtering tools.

No Unified Interface:

No single platform allows filtering by genre, content type, and user rating simultaneously.

Data Not Leveraged:

Rich relational data across movies, categories, and user preferences remains unexplored.



How do we design a database and queries to solve these challenges?

SCHEMA

Database Schema Overview

Three tables organize movies, categories, and user preferences.

Core Tables

- **movies** — movie_id, title, type, category_id
- **categories** — category_id, category_name

Preferences Table

- **preferences** — preference_id, movie_id, rating

Solution Approach



Step 1 - Design 3-Table Schema

Create Movies, Categories, and Preferences tables with proper relationships.



Step 2 - Load Sample Data

Use the provided movie dataset with titles like Inception, Breaking Bad, and 3 Idiots, along with genres and ratings.



Step 3 - Build Tkinter GUI

Create filter dropdowns for Genre, Type, and Min Rating with a search button.



Step 4 - Query & Display

Execute filtered SQL queries and display results in a table view.

SQL

Recommendation Approaches



Genre Filter

Filter movies by category: Action, Comedy, Drama, Sci-Fi, Romance, or Thriller.



Type Filter

Show only Movies or Series depending on the user's selection.



Rating Filter

Limit results by minimum rating, from 0 to 5 stars.

Database Connection & Query Logic

Connection Setup

MySQL connector for database access

Connection details: host, user, password, and database

Reusable function: `get_db_connection()` keeps the connection logic in one place

Python Code

```
import mysql.connector

def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="tulika",
        database="streaming_db"
    )

conn = get_db_connection()
cursor = conn.cursor()
```

Query 2 — Filtered Search (Genre, Type, Rating)

What it does

Filters movies by selected genre

Filters by content type (Movie or Series)

Filters by minimum rating threshold

Joins Movies, Categories, and Preferences tables

Python Code

```
query = """
SELECT m.title, m.type, c.category_name,
IFNULL(p.rating, 'N/A')
FROM Movies m
LEFT JOIN Categories c ON m.category_id = c.category_id
LEFT JOIN Preferences p ON m.movie_id = p.movie_id
WHERE c.category_name = %s
      AND m.type = %s
      AND (p.rating >= %s OR p.rating IS NULL)
"""
cursor.execute(query, (genre, type, rating))
```

PYTHON

Query 3 — Tkinter GUI Implementation

Step 1 - Filter Controls

Create dropdowns for Genre, Type, and Min Rating with a Search button.

Step 2 - Execute Query

Call `update_table()` to fetch filtered results from MySQL.

Step 3 - Display Results

Show results in a Treeview table with Title, Type, Genre, and Rating columns.

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Movie Library Explorer")
root.geometry("700x500")

frame = ttk.Frame(root, padding="20")
frame.pack(fill='x')

ttk.Label(frame, text="Genre:").grid(row=0, column=0)
genre_combo = ttk.Combobox(frame, values=["Action", "Comedy", "Drama", "Sci-Fi", "Romance", "Thriller"])
genre_combo.grid(row=0, column=1)

ttk.Label(frame, text="Type:").grid(row=0, column=2)
type_combo = ttk.Combobox(frame, values=["Movie", "Series"])
type_combo.grid(row=0, column=3)

ttk.Button(frame, text="Search Library", command=update_table).grid(row=0, column=6)

tree = ttk.Treeview(root, columns=('Title', 'Type', 'Genre', 'Rating'), show='headings')
tree.pack(pady=20, fill='both', expand=True)

root.mainloop()
```


Summary & Next Steps

Key Takeaways

3-Table Relational Schema

Movies, Categories, and Preferences tables with proper foreign key relationships.

MySQL + Python Integration

MySQL connector bridges the database and Python application seamlessly.

Interactive Tkinter GUI

User-friendly interface with dropdown filters and live search results.

Extensible Design

Easy to add more filters, sorting options, or recommendation features.

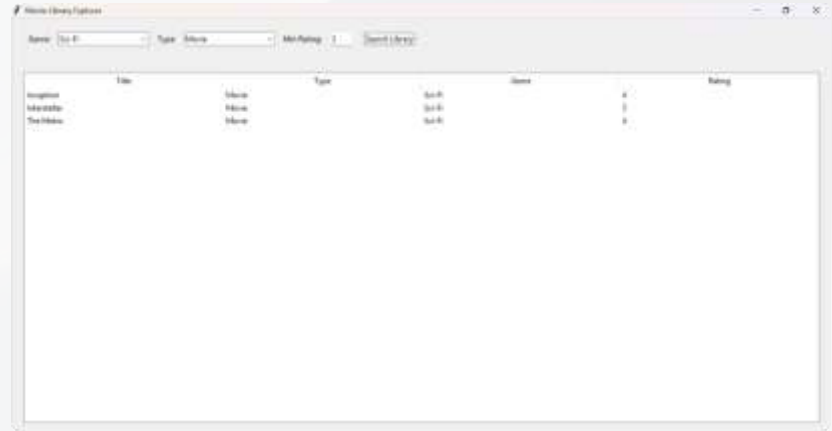
A well-designed database and intuitive UI transform raw data into actionable insights.

OUTPUT

Application Output



Left: Empty application window before filtering



Right: Application with filtered results showing Inception, Breaking Bad, 3 Idiots

Database Schema (Visual)

Categories Table

category_id INT PK
category_name
VARCHAR(50)
Example: Action, Comedy,
Drama, Sci-Fi, Romance, Thriller

Movies Table

movie_id INT PK
title VARCHAR(50)
category_id INT FK
type VARCHAR(20)
Example: 13 movies — Movies &
Series, Hollywood & Bollywood

Preferences Table

pref_id INT
movie_id INT
rating INT
Example: 12 user ratings —
Related to Categories

Thank You

Tulika Agrawal

3-Table Relational DB with FK constraints

Python GUI with live filtered search

Parametrized queries for safe, flexible filtering